

PATENT APPLICATION

SYSTEM AND METHOD FOR OFFLINE SEMANTIC PRE-COMPUTATION AND KNOWLEDGE GRAPH CONSTRUCTION FROM TIMESTAMPED TEXT CORPORA

Inventor: Eric G. Suchanek, PhD
Assignee: Flux-Frontiers
Filing Date: March 26, 2026
Application Type: Utility Patent

CROSS-REFERENCE TO RELATED APPLICATIONS

Not applicable.

FIELD OF THE INVENTION

The present invention relates generally to natural language processing and knowledge graph construction, and more particularly to a system and method for offline semantic pre-computation of timestamped text corpora that eliminates query-time inference while preserving full semantic and temporal search capability.

BACKGROUND OF THE INVENTION

The Prior Art and Its Deficiencies

Modern semantic search systems operate under two foundational assumptions that constrain their deployment, cost, and privacy characteristics.

First assumption: semantic understanding requires live inference. When a user queries a document corpus, conventional systems invoke a large language model (LLM) at query time — either via cloud API or local GPU inference — to process the

query and generate semantically relevant results. This architecture imposes per-query costs that scale linearly with query volume, requires persistent network connectivity for cloud-based systems, and demands specialized hardware (GPU clusters) for local deployment. Consumer-grade hardware cannot serve these workloads at interactive latencies for corpora exceeding trivial sizes.

Second assumption: temporal grounding requires LLM extraction. Dates, time references, and temporal relationships embedded in natural language prose are conventionally extracted by language models at either ingest or query time. For locally-deployed small-parameter models (e.g., 4 billion parameters), this extraction fails systematically — such models hallucinate dates, confuse temporal formats, produce malformed output, and cannot reliably distinguish relative from absolute time references. This failure mode blocks local, offline execution of any temporally-aware memory or search system.

Prior art systems addressing semantic search over document corpora include Retrieval-Augmented Generation (RAG) pipelines, which embed documents into vector stores but require LLM inference at query time for answer generation; traditional knowledge graph systems, which provide structural querying but lack integrated semantic vector search; and embedding-based search engines, which provide semantic similarity but lack temporal provenance, topic classification, and structural graph relationships.

No existing system provides, in a single integrated pipeline executable on consumer hardware without network connectivity: (a) multi-phase NLP enrichment with configurable topic and context classification; (b) parallel multi-process vector embedding with no external service dependency; (c) a hybrid knowledge graph combining structural indices with semantic vector indices; (d) temporal provenance on every node written directly from source metadata without LLM extraction; and (e) sub-three-minute full-corpus ingestion at the scale of thousands of entries.

SUMMARY OF THE INVENTION

The present invention provides a system and method for constructing a semantically enriched, temporally grounded knowledge graph from timestamped text corpora through offline pre-computation. The core innovation is a shift from query-time inference to ingest-time pre-computation: all semantic understanding — topic labeling, context classification, vector embedding, and graph edge construction — is computed once during corpus ingestion. The resulting knowledge graph is a frozen, versioned,

queryable artifact that serves all subsequent queries through vector lookup and graph traversal alone, with no model loading or inference at query time.

The system comprises six sequential processing stages orchestrated by a unified pipeline controller:

1. A multi-phase NLP transformer that parses timestamped source entries, applies configurable sentence-group chunking, performs multi-label topic classification and single-label context classification using domain-configurable YAML vocabularies, and supports temporally-diverse sampling;
2. A corpus emitter that writes each semantically enriched chunk as a structured document with metadata frontmatter, embedding temporal provenance directly from parsed source timestamps without any language model extraction;
3. A structural graph builder that indexes all chunks into a SQLite database with BM25 lexical search capability and typed graph edges;
4. A semantic vector index builder that generates dense vector embeddings and stores them in a columnar vector database for approximate nearest-neighbor search;
5. A multi-process parallel embedder that shards the corpus across CPU worker processes, each loading an independent model instance with no shared state and no GIL contention; and
6. A JSON embedding cache that stores aligned arrays of embeddings, texts, and timestamps for downstream analysis.

The invention inverts the conventional cost model: ingestion cost is medium and paid once; query cost is negligible and does not increase with query volume; privacy is preserved because no data leaves the local machine; and offline operation is fully supported.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

1. System Overview

Referring now to the invention in detail, the system operates on timestamped text corpora in a pipe-delimited enriched format:

TIMESTAMP	TYPE	CATEGORY	CONTENT
-----------	------	----------	---------

Each entry carries a temporal anchor (an ISO-format timestamp), a semantic type label, and a context category. This format is produced by a preprocessing layer that enriches raw text with natural language processing — named-entity recognition, sentence segmentation, and domain-specific topic classification via configurable vocabularies.

The system has been reduced to practice and validated on a corpus of 6,450 diary entries spanning 9.6 years (1660–1669), comprising 23,235 enriched lines totaling 3.3 megabytes. Full pipeline execution completes in approximately three minutes on consumer-grade Apple Silicon hardware with no GPU and no network connectivity.

2. Stage 1: Multi-Phase NLP Transformation

The first processing stage applies five sequential NLP phases to the enriched source text:

Phase 1 — Parsing. The system reads pipe-delimited lines and validates ISO timestamps. Malformed entries are logged and excluded. Each valid entry produces a structured record with timestamp, type, category, and content fields.

Phase 2 — Sentence-Group Chunking. The content of each entry is segmented into chunks using a configurable chunking strategy. The preferred embodiment uses sentence-group chunking with a default of four sentences per chunk and a maximum of 512 characters. Alternative strategies include hybrid chunking (combining sentence boundaries with character limits) and semantic chunking (grouping sentences by topical coherence). The chunking parameters are configurable to accommodate different corpus characteristics.

Phase 3 — Multi-Label Topic Classification. Each chunk is classified against a domain-configurable YAML vocabulary that maps topic labels to keyword lists. Classification uses TF-IDF weighted keyword matching. Each chunk may carry multiple topic labels (e.g., a single chunk may be labeled with both “domestic” and “court” topics). The vocabulary is fully user-configurable, enabling domain adaptation without code changes.

Phase 4 — Single-Label Context Classification. Each chunk receives exactly

one context label from a second configurable YAML vocabulary. Context categories represent the situational frame of the content (e.g., “Office,” “Home,” “Social,” “Reflection,” “Health”). Assignment uses keyword matching with priority ordering defined in the vocabulary.

Phase 5 — Temporally-Diverse Sampling. When a batch size is specified (for processing a subset of a large corpus), the system applies temporally-diverse sampling using uniform stride across the time axis rather than random sampling. This ensures representative coverage across the full temporal span of the corpus, preventing front-loading bias that would result from naive sequential or random sampling.

Parallelism. The NLP transformation stage accepts a configurable `workers` parameter that distributes feature extraction across CPU cores via process-level parallelism. Each worker processes an independent shard of the corpus with no shared state, enabling near-linear scaling with available CPU cores.

3. Stage 2: Corpus Emission with Direct Temporal Provenance

Each semantically enriched chunk is written as a Markdown file with YAML frontmatter containing structured metadata:

```
---
source_file: corpus/enriched_full.txt
timestamp: 1666-09-02T08:00
topic: domestic
context: Home
chunk_index: 0
---
[chunk text content]
```

Listing 1: Emitted chunk document with YAML frontmatter

Critical innovation: direct temporal database writes. The timestamp in the frontmatter is written directly from the parsed ISO timestamp in the source format. No language model is involved in temporal grounding at any stage. This completely bypasses the temporal extraction failure mode that plagues small-parameter local models, making the temporal dimension fully reliable regardless of model capability. The temporal provenance is a first-class metadata field on every chunk, available for filtering, sorting, and graph construction in all downstream stages.

4. Stage 3: Structural Knowledge Graph Construction

The system delegates to a structural graph builder that indexes all emitted chunk files into a SQLite database. The structural index provides:

BM25 Lexical Search. Full-text search over chunk content using the Okapi BM25 ranking function. This supports exact term matching, phrase queries, and Boolean combinations.

Typed Graph Edges. The system constructs typed edges between nodes:

- CONTAINS edges linking source documents to their constituent chunks;
- SIMILAR edges linking chunks with high lexical overlap;
- TEMPORAL edges linking chunks that are temporally adjacent in the source corpus.

Metadata Filtering. All frontmatter fields (timestamp, topic, context, source file, chunk index) are indexed as filterable metadata columns, enabling efficient predicate pushdown for combined structural and metadata queries.

The validated structural graph for the reference corpus contains 29,402 nodes and 355,250 edges in a 109 MB SQLite database.

5. Stage 4: Semantic Vector Index Construction

The system builds a semantic vector index using dense embeddings from a pre-trained sentence transformer model. In the preferred embodiment, the model is `all-mpnet-base-v2`, producing 768-dimensional float32 vectors.

The vector index is stored in a columnar vector database (LanceDB in the preferred embodiment) optimized for approximate nearest-neighbor (ANN) search. The index supports cosine similarity queries with sub-millisecond latency.

Critical design point: All embedding computation occurs during this build stage. At query time, the vector index serves pre-computed embeddings directly. No model is loaded and no inference is performed to answer a query.

The validated vector index for the reference corpus occupies 102 MB for 6,647 chunk embeddings at 768 dimensions.

6. Stage 5: Multi-Process Parallel Embedding

The embedding pipeline operates independently of the knowledge graph build, producing an embedding cache optimized for analysis:

```
Parse -> (optional temporal sample) -> shard across workers
    -> each worker:
        load sentence-transformer model locally -> encode shard
    -> return float32 array
-> concatenate shards in original order -> write cache
```

Listing 2: Multi-process parallel embedding pipeline

Key design decisions providing the inventive step:

Independent model instances per worker. Each worker process loads its own copy of the sentence-transformer model. There is no shared memory between workers and no Global Interpreter Lock (GIL) contention. This is in contrast to conventional approaches that share a single model instance across threads, which in Python-based systems creates GIL contention that prevents true parallelism.

Spawn start method enforcement. On POSIX systems, the system enforces the `spawn` start method for subprocess creation rather than the default `fork` method. This ensures clean subprocess isolation, preventing issues with inherited file descriptors, CUDA contexts, or other process state that can cause subtle failures in forked multiprocessing.

Temporally-uniform sampling. When processing a subset of the corpus, temporal sampling uses uniform stride across the time axis rather than random sampling, ensuring that the embedding cache provides representative manifold coverage of the full temporal span.

Aligned output format. The output consists of three aligned arrays — embeddings, texts, and timestamps — stored as JSON. This self-contained format requires no database dependency and is portable across environments.

The architecture achieves near-linear scaling with CPU count on encode-bound workloads. In the reference implementation, four worker processes embed 6,647 chunks as part of a full pipeline execution completing in approximately three minutes.

7. Stage 6: Embedding Cache

The JSON embedding cache stores three parallel arrays:

```
{
  "embeddings": [[float32, ...], ...],
  "texts": ["chunk text", ...],
  "timestamps": ["ISO timestamp", ...]
}
```

Listing 3: Embedding cache JSON structure

The arrays are index-aligned: `embeddings[i]`, `texts[i]`, and `timestamps[i]` all correspond to the same source chunk. This format is intentionally simple, requiring no database dependency and supporting direct consumption by any numerical computing environment.

8. Unified Pipeline Orchestration with Snapshot System

The system provides a unified orchestrator that executes the full pipeline — NLP transformation, corpus emission, structural graph indexing, vector index construction, and snapshot capture — through a single invocation.

Point-in-time snapshots capture corpus metrics (entry count, chunk count, temporal span), graph metrics (node count, edge count, index sizes), and build metadata (timestamp, hardware, duration). Snapshots are versioned and enable comparison across corpus rebuilds, supporting reproducibility and regression detection.

9. Hybrid Retrieval Without Query-Time Inference

The completed knowledge graph supports hybrid retrieval combining:

- **BM25 lexical search** for exact term matching and Boolean queries;
- **Vector ANN search** for semantic similarity queries;
- **Metadata filtering** for temporal range queries, topic filtering, and context filtering;
- **Graph traversal** for structural navigation (containment, similarity, temporal adjacency).

All four retrieval modalities operate on pre-computed indices. No model is loaded at query time. No inference is performed. Query latency is sub-millisecond for the reference corpus of 29,402 nodes.

The following table summarizes the validated reference corpus metrics:

Metric	Value	Notes
Source entries	6,450	Diary entries, 1660–1669
Enriched lines	23,235	Pipe-delimited format
Source file size	3.3 MB	
Chunk count	6,647	Sentence-group chunking
Graph nodes	29,402	
Graph edges	355,250	CONTAINS, SIMILAR, TEMPORAL
SQLite database size	109 MB	Structural index
LanceDB vector index	102 MB	768-dimensional float32
Combined KG footprint	~241 MB	
Embedding dimensions	768	<code>all-mpnet-base-v2</code>
Full pipeline duration	~3 min	Apple Silicon, no GPU
Query latency	<1 ms	Pre-computed indices

CLAIMS

Independent Claims

Claim 1. A computer-implemented method for constructing a queryable knowledge graph from a timestamped text corpus, comprising:

- (a) receiving a plurality of text entries, each text entry comprising a timestamp, a semantic type label, a context category, and natural language content;
- (b) for each text entry, applying a multi-phase natural language processing transformation comprising:
 - (i) parsing and validating the timestamp;
 - (ii) segmenting the content into one or more chunks according to a configurable chunking strategy;
 - (iii) classifying each chunk with one or more topic labels using a configurable topic vocabulary;
 - (iv) classifying each chunk with a single context label using a configurable context vocabulary;
- (c) emitting each chunk as a structured document file comprising

the chunk text and metadata frontmatter, wherein the metadata frontmatter includes a temporal provenance field written directly from the parsed timestamp of step (b)(i) without invoking any language model for temporal extraction;

- (d) constructing a structural graph index from the emitted document files, the structural graph index comprising nodes representing chunks, typed edges representing containment, similarity, and temporal adjacency relationships, and a full-text search index;
- (e) constructing a semantic vector index by computing dense vector embeddings for each chunk using a pre-trained sentence-transformer model and storing the embeddings in a columnar vector store supporting approximate nearest-neighbor search;

wherein steps (d) and (e) collectively constitute ingest-time pre-computation such that all subsequent queries against the knowledge graph are served by vector lookup and graph traversal without loading any language model or performing any neural network inference at query time.

Claim 2. A computer-implemented method for parallel vector embedding of a text corpus without shared state, comprising:

- (a) receiving a plurality of text chunks derived from a timestamped text corpus;
- (b) partitioning the text chunks into a plurality of shards, one shard per available CPU core;
- (c) spawning a plurality of worker processes using a spawn start method, wherein each worker process:
 - (i) loads an independent instance of a sentence-transformer model into its own process memory;
 - (ii) encodes its assigned shard of text chunks into dense vector embeddings;
 - (iii) returns the embeddings as a contiguous float32 array;
- (d) concatenating the returned embeddings from all worker processes in the original corpus order;

wherein no shared memory is used between worker processes, no Global Interpreter Lock contention occurs, and the method achieves near-linear

scaling with the number of available CPU cores.

Claim 3. A system for offline semantic pre-computation of a timestamped text corpus, comprising:

- (a) a multi-phase NLP transformer module configured to parse timestamped entries, segment content into chunks, and classify each chunk with topic and context labels from user-configurable YAML vocabularies;
- (b) a corpus emitter module configured to write each chunk as a structured document file with metadata frontmatter containing temporal provenance written directly from source timestamps without language model extraction;
- (c) a structural graph builder module configured to index the document files into a relational database comprising nodes, typed edges, and a BM25 full-text search index;
- (d) a semantic vector index builder module configured to compute dense vector embeddings using a pre-trained sentence-transformer model and store the embeddings in a columnar vector database;
- (e) a multi-process parallel embedder module configured to shard the corpus across worker processes, each worker loading an independent model instance with no shared state;
- (f) a unified orchestrator configured to execute modules (a) through (e) in sequence through a single invocation and to capture point-in-time snapshots of corpus and graph metrics;

wherein the system is configured to execute entirely on consumer-grade CPU hardware without GPU acceleration and without network connectivity, and wherein all queries against the constructed knowledge graph are served without loading any language model or performing any inference.

Dependent Claims

Claim 4. The method of Claim 1, wherein the configurable chunking strategy of step (b)(ii) comprises sentence-group chunking with a configurable number of sentences per chunk and a configurable maximum character count per chunk.

- Claim 5.** The method of Claim 1, wherein the multi-label topic classification of step (b)(iii) uses TF-IDF weighted keyword matching against a YAML vocabulary file that maps topic labels to keyword lists, and wherein each chunk may receive zero, one, or multiple topic labels.
- Claim 6.** The method of Claim 1, further comprising applying temporally-diverse sampling when processing a subset of the corpus, wherein the sampling uses uniform stride across the time axis of the corpus to ensure representative temporal coverage.
- Claim 7.** The method of Claim 1, wherein the structural graph index of step (d) comprises:
- CONTAINS edges linking source document nodes to constituent chunk nodes;
 - SIMILAR edges linking chunk nodes having lexical overlap exceeding a configurable threshold;
 - TEMPORAL edges linking chunk nodes that are temporally adjacent in the source corpus.
- Claim 8.** The method of Claim 1, wherein the semantic vector index of step (e) uses a 768-dimensional embedding space generated by a pre-trained sentence-transformer model, and wherein the columnar vector store supports cosine similarity approximate nearest-neighbor queries with sub-millisecond latency.
- Claim 9.** The method of Claim 2, wherein the spawn start method is enforced on POSIX systems to ensure clean subprocess isolation and to prevent inheritance of file descriptors, GPU contexts, or other parent process state.
- Claim 10.** The method of Claim 2, further comprising, prior to step (b), applying temporally-uniform sampling using stride-based selection across the time axis of the corpus.
- Claim 11.** The system of Claim 3, further comprising a snapshot module configured to capture point-in-time metrics including entry count, chunk count,

temporal span, node count, edge count, index sizes, and build metadata, and to support comparison between snapshots captured at different build times.

Claim 12. The system of Claim 3, wherein the structural graph builder module stores the relational database as a SQLite file and the semantic vector index builder module stores the vector database as a LanceDB directory, and wherein the combined knowledge graph footprint for a corpus of approximately 6,500 entries is approximately 241 megabytes.

Claim 13. The method of Claim 1, wherein the knowledge graph supports hybrid retrieval combining BM25 lexical search, vector approximate nearest-neighbor search, metadata filtering on temporal range and topic and context labels, and graph traversal across typed edges, all without loading any model or performing any inference at query time.

Claim 14. The method of Claim 1, wherein the entire pipeline from raw corpus ingestion to completed knowledge graph executes in less than five minutes for a corpus of up to 10,000 entries on consumer-grade hardware without GPU acceleration.

Claim 15. The system of Claim 3, wherein the user-configurable YAML vocabularies of module (a) enable domain adaptation to arbitrary timestamped text corpora — including personal journals, clinical notes, legal case files, corporate meeting transcripts, research literature, and biographical archives — without modification to any system module.

ABSTRACT

A system and method for constructing a semantically enriched, temporally grounded knowledge graph from timestamped text corpora through offline pre-computation. The system applies multi-phase NLP transformation — parsing, chunking, multi-label topic classification, and context classification using domain-configurable YAML vocabularies — to produce structured document chunks with temporal provenance written directly from source timestamps without any language model extraction. The chunks are indexed into a hybrid knowledge graph comprising a structural relational database with BM25 full-text search and typed graph edges, and a semantic vector index with

dense embeddings from a pre-trained sentence-transformer model. A multi-process parallel embedding pipeline shards the corpus across CPU workers, each loading an independent model instance with no shared state, achieving near-linear scaling. A unified orchestrator executes the full pipeline through a single invocation with point-in-time snapshot capture. The resulting knowledge graph serves all subsequent queries — lexical, semantic, metadata, and structural — through pre-computed indices with sub-millisecond latency, no model loading, and no inference at query time. The system executes entirely on consumer-grade CPU hardware without GPU acceleration or network connectivity.

Patent Application – Eric G. Suchanek, PhD – Flux-Frontiers – March 26, 2026